Technische Hochschule Deggendorf
Faculty of Applied Informatics
Programme Artificial Intelligence BSc

QoE Prediction from Encrypted Traffic with MARINA on P4/Tofino

Bachelor Thesis

Bachelor of Science (B.Sc.)

at the Technische Hochschule Deggendorf

Submitted by:
Elmer Dema
Matriculation number: 22211551
elmer.dema@stud.th-deg.de /
elmerdema2022@gmail.com

Examiner / Supervisor:
Prof. Dr. Andreas Kassler

Semester: WS2025
Submission date: 2026-03-16

DEGGENDORF INSTITUTE OF TECHNOLOGY

Declaration of Authorship

Name of student: Elmer Dema

Name of supervisor: Prof. Andreas Kassler

Thesis topic:

QoE Prediction from Encrypted Traffic with MARINA on P4/Tofino

1. I hereby declare that the thesis has been written in compliance with Section 35 (7) RaPO (State Examination Regulations in BavariaBayRS 2210-4-1-4-1-WFK) is my own work, has not been submitted for any other degree at any other university or institution, does not contain or use any sources or resources other than those referenced, and that all direct and paraphrased quotes have been duly cited as such.

Deggendorf, 03/16/2026
(Date)

(Signature of student)

2. I hereby consent to my completed thesis being made available to a broader public through the library of Deggendorf Institute of Technology:

x No
Yes, upon completion of the examination process.
Yes, upon expiry of a non-disclosure period of _____ years.

I certify and warrant that I am the sole owner and holder of all rights to the thesis, including the right to use the tables, figures, illustrations, plans, or similar, submitted therein and that no third-party rights or claims, nor any statutory provisions, will be infringed as a result of its publication.

Deggendorf, 03/16/2026
(Date)

(Signature of student)

To be completed by the supervisor if the author consents to the thesis being published:

Collection of a copy of the thesis by the library, including the lending of the copy, is hereby

endorsed,
x not endorsed.

Deggendorf, 03/16/2026
(Date)

(Signature of supervisor)

Acknowledgements

*I would like to express my sincere gratitude to my professor, **Andreas Kassler**, for his invaluable guidance and support throughout this thesis.*

*I am also deeply thankful to my lab partner **Juled Zaganjori** and **Lukas Froschauer** (research scientist), for their collaboration.*

*Special thanks to **Jonathan Langlet** and the **Pforest team** for providing the initial translator, reporter code, and pforest architecture, which were instrumental in the successful completion of this work.*

*I would also like to acknowledge my sister, **Elma**, for her unwavering support.*

Elmer Dema
March 2026

"You know, Quasimodo predicted all of this"
— B.B.

Abstract

Quality of Experience (QoE) is a crucial metric for multimedia services, as it directly reflects user satisfaction with platforms such as YouTube or Netflix. However, measuring QoE is challenging due to its subjective nature and the difficulty of obtaining real-time user feedback. Traditional methods relying on surveys or manual inputs are not scalable for large-scale network environments.

While papers, such as ViCrypt , have demonstrated that network statistics can predict user QoE, its reliance on end-host GPUs for processing features makes it impossible for direct implementation in the network data plane [1]. To bridge this critical gap, this thesis addresses the following research question: *How can Machine Learning models for QoE prediction be efficiently implemented directly within the network data plane to provide real-time monitoring of encrypted video traffic, while navigating the fundamental trade-offs between predictive accuracy and hardware resource constraints?* This work hypothesizes that by using time-windowed rolling statistics and mapping a constrained Random Forest classifier into Ternary Match-Action Tables, it is possible to perform accurate, line-rate QoE prediction strictly within the physical memory and pipeline constraints of the Intel Tofino ASIC.

In order to achieve this, a hybrid architecture is introduced for real-time QoE monitoring that performs line-rate feature extraction and prediction. This design divides the predictive model into two components: a module deployed on a P4/Tofino switch performs initial video flow classification and extracts primitive features at line rate, while the final QoE prediction is made using a lightweight machine learning model on the switch itself. The main contributions include a mapping of Random Forest classifiers to P4 match-action tables and a comprehensive benchmarking framework for evaluating in-network ML performance.

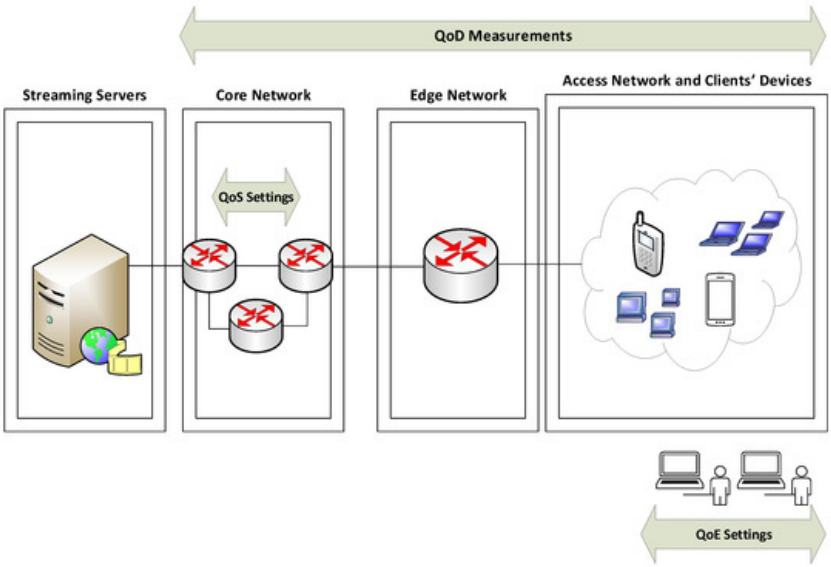


Figure 1: **a network architecture diagram illustrating the data flow from streaming servers to client devices, while highlighting specific stages for measuring Quality of Service (QoS), Quality of Delivery (QoD), and Quality of Experience (QoE). Source**

Contents

ACKNOWLEDGEMENTS . . . . . **I**

ABSTRACT . . . . . **II**

1. INTRODUCTION **1**

2. RELATED WORK **3**
2.1. Quality of Experience Modeling 3
2.2. In-Network Machine Learning 3
2.3. Benchmarking and Hardware Constraints 4

3. DESIGN & METHODOLOGY **5**
3.1. REPORTER System Architecture 5
3.2. Data Plane Translator Architecture 6
3.2.1. Telemetry Parsing and Feature Extraction 6
3.2.2. Random Forest Hardware Mapping 6
3.2.2.1. Ternary Range Matching (TCAM) 7
3.2.2.2. Physical Stage Unrolling 7
3.2.3. Voting Mechanism 7
3.2.4. State-Dependent Routing and Deparsing 8
3.3. Evolution of the Methodology 8
3.3.1. Advanced Feature Engineering: Rolling Statistics 9
3.3.2. Target Class Definition: Risk-Based QoE State 9
3.4. Preliminary Modeling and Feature Importance 10
3.4.1. Performance Evaluation 10
3.4.2. Feature Importance Analysis 11

4. IMPLEMENTATION **12**
4.1. Dataset Analysis 12
4.1.1. Data Ingestion and Parsing 12
4.1.2. Synchronization and Preprocessing 13
4.1.3. Feature Engineering and Windowing 13
4.1.4. Target Alignment and Dataset Construction 15
4.2. Model Training via pForest 16
4.2.1. The Training Pipeline (`rf_model.py`) 16
4.2.2. Deviation from Original pForest Architecture 17
4.2.3. Automated P4 Code Generation (`p4_generator_tna.py`) 17
4.2.4. Control Plane Configuration: Forwarding 18
4.2.5. Isolated Forwarding Table Configuration 18
4.2.6. BFRT gRPC Implementation 19
4.2.7. Table Logic and Topology Integration 19
4.2.8. Full Control Plane Script 21

5. EVALUATION **24**
5.1. System Evaluation and Translator Testing 24

5.1.1. Phase 1: Telemetry Data Generation (Reporter) 24
5.1.2. Phase 2: Translator Inference Testing 24
5.1.3. Packet Manipulation and Topology Adaptation 24
5.1.4. Timing Preservation 25
5.1.5. Overcoming Testbed Artifacts: MAC Learning 25
5.1.6. Execution and Capture 26
5.1.7. Telemetry Generation Results 26
5.2. Benchmarking and Performance Analysis 28
5.2.1. Feature Scaling and Hardware Constraints 28
5.2.2. Tree and Depth Configuration Sweep 29
5.2.3. Feature Importance and Resource Overview 30
5.3. Conclusion 31

LITERATURE AND BIBLIOGRAPHY 32

LIST OF FIGURES XXXIV

LIST OF TABLES XXXV

LIST OF FORMULAS XXXVI

LIST OF ABBREVIATIONS XXXVII

1. Introduction

Machine Learning (ML) is a subgroup of Artificial Intelligence (AI), a field where computers autonomously learn from past data to generate predictions or solutions. Recently, there has been a surge in applying ML methods to address practical challenges within computer networks [2]. This thesis implements and extends recent applications of ML techniques that use Quality of Experience (QoE) metrics to predict video quality. These QoE measurements are essential for capturing the end-to-end performance of network services [3]. As video data makes up the majority of global IP traffic [4], network operators are under pressure to optimize resource management while satisfying user demands. In general, Video Quality Assessment (VQA) has relied on both subjective [5] and objective [6] methodologies. Subjective methods focus on quantifying the end-user's satisfaction. Consequently, network traffic management driven by Quality of Experience (QoE) depends on monitoring and forecasting application-level performance or QoE via video Key Performance Indicators (KPIs) that impact the user. The International Telecommunication Union (ITU) characterizes QoE as the overall experience of a service from the subjective viewpoint of the end user [7].

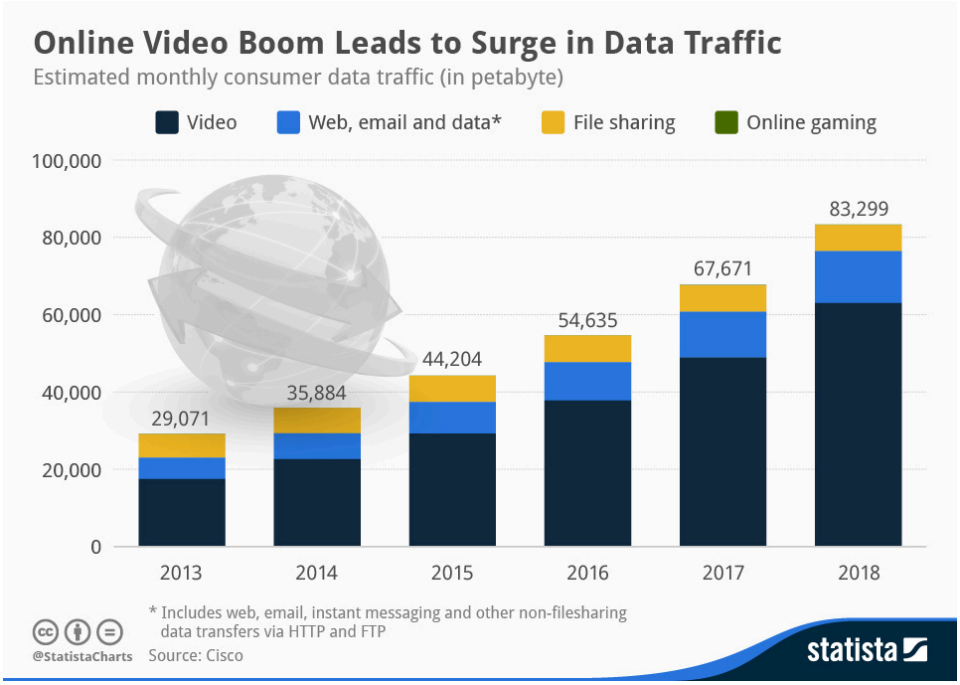


Figure 2: **Video effect of the devices on traffic is more pronounced because of the introduction of Ultra-High-Definition (UHD), or 4K, video streaming. Source**

A significant obstacle in video streaming is the absence of a standardized method[1] for measuring QoE [8], which has stimulated research into QoE models derived from network statistics.

Such modeling assists in defining specific KPIs for various user categories. For example, network providers might analyze tools like rebuffering or quality switching to understand their impact on the video stream.

This helps analyze the effect on user QoE and correlates these issues with network parameters such as jitter or delay. While subjective studies offer reliable evaluations, they are noted for being expensive, time intensive and impractical for real-world applications [9]. Meanwhile, objective VQA uses mathematical models to estimate perceived quality. Although metrics such as Peak Signal-to-Noise Ratio (PSNR) [10] and Structural Similarity Index (SSIM) are computationally efficient, they do not always accurately mirror the actual user experience [11], [12].

The main focus in this thesis is on network performance rather than direct user experience. Since we are operating in the switch, the focus is on network-layer parameters instead of application layers, with QoE estimated from buffering-related events (e.g., stalling and depletion).

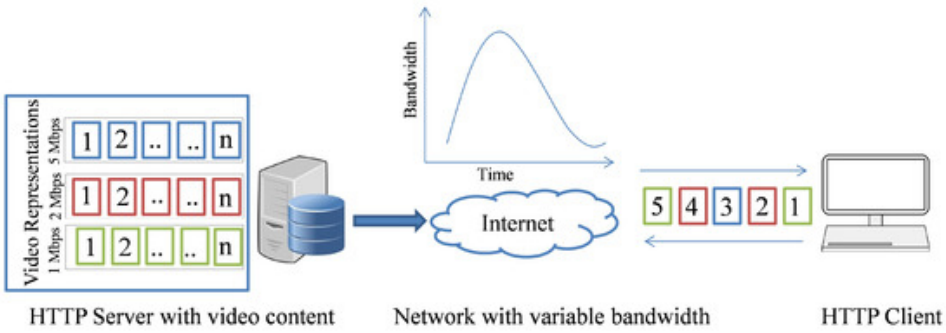


Figure 3: **MPEG DASH is a scenario where a video player dynamically switches between different quality levels of media segments delivered over HTTP to adapt to fluctuating network conditions for smooth playback.**

[1]The P.1203 model was validated only for certain content conditions (e.g., video up to 1080p and 25 fps). Extrapolating beyond these ranges (e.g., 4K, high fps content) may create unreliable quality estimates unless updated or extended versions are used.

2. Related Work

2.1. Quality of Experience Modeling

Traditional approaches to QoE modeling have relied on subjective user studies and objective mathematical models to quantify end-user satisfaction [6], [7]. However, with the adoption of end-to-end encryption (e.g., HTTPS, QUIC) for multimedia delivery, Deep Packet Inspection (DPI) and payload-based video quality analysis are no longer viable. Consequently, recent literature has pivoted towards inferring QoE metrics directly from encrypted network traffic patterns.

Frameworks such as ViCrypt [1] have demonstrated the feasibility of using Machine Learning (ML) to predict video QoE from network-layer features. Further extending this, contemporary studies have utilized ML models to estimate key QoE indicators for encrypted DASH traffic in modern cellular networks [13] and live video conferencing platforms [14]. These works collectively establish that statistical network properties such as packet sizes, inter-arrival times (IAT), and jitter are strong predictors of application-level events like buffer depletion and stalling, forming the theoretical basis for the feature engineering in this thesis.

2.2. In-Network Machine Learning

While ML-based QoE prediction is well-established, relying on end-host GPUs or external collector servers introduces latency and limits scalability in high-throughput environments. Recent advancements in programmable data planes, specifically using the P4 language, have enabled the displacement of some monitoring and decision-making logic from the control plane to the data plane [15].

Executing complex ML models like Random Forests at line rate, however, requires specialized hardware mapping. The pForest architecture [16] introduced the concept of translating context-dependent decision trees into Match-Action Tables (MATs) for in-network inference. Building upon this paradigm, frameworks like Planter [17] have formalized the automated mapping of ensemble ML models into programmable switches. Other recent works have explored resource-efficient implementations of tree-based models using feature sharing and knowledge distillation [18]. These methodologies provide the architectural foundation for our approach of translating a unified Random Forest model into unrolled physical pipeline stages using Ternary Content-Addressable Memory (TCAM).

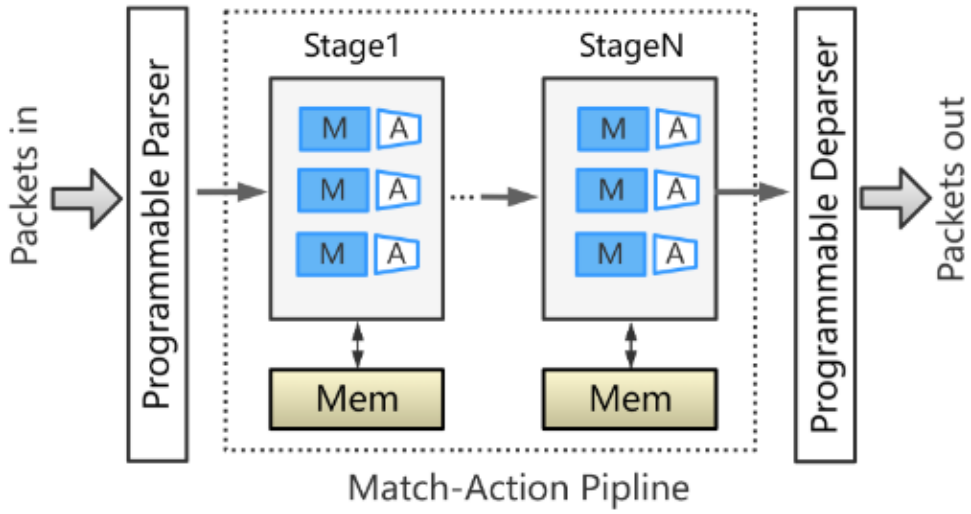


Figure 4: **The general architecture of a P4 switch. [18]**

2.3. Benchmarking and Hardware Constraints

Evaluating the performance of ML models on hardware such as the Intel Tofino requires a deep understanding of its strict physical constraints. As highlighted in recent comprehensive surveys on in-network machine learning [19], programmable Application-Specific Integrated Circuits (ASICs) are fundamentally bottlenecked by available SRAM, TCAM, Very Long Instruction Word (VLIW) instruction slots, and a hard limit on physical pipeline stages.

Studies evaluating hybrid in-network classification frameworks, such as IIsy [20], emphasize the complex multi-dimensional trade-offs between an ML model's predictive accuracy and its hardware resource footprint. Furthermore, because the Tofino architecture enforces a strict Directed Acyclic Graph (DAG) for table execution, recursive branching inherent in standard decision trees must be aggressively flattened [21]. These studies guide our benchmarking approach, showing that feature selection, limited tree depth, and tuning are essential to avoid compiler timeouts and hardware resource exhaustion.

3. Design & Methodology

3.1. REPORTER System Architecture

The foundation of the solution builds upon the Reporter[2] framework, a programmable switch architecture designed for real-time traffic classification. As illustrated in Figure 5, the system operates on a decoupled **Control Plane** and **Data Plane** model to handle high-throughput video traffic.

The processing flow begins in the Data Plane when raw packets enter the ingress pipeline. For every incoming packet, the REPORTER updates Feature Registries with new telemetry data. A subset of these extracted features is then used by a **Random Forest (RF) Classifier**, which is encoded directly into the reporter's hardware. This classifier determines if a packet belongs to a video flow.

If a packet is classified as a video packet, the REPORTER sends a **telemetry packet** to a downstream Translator Switch at a specific time interval. This telemetry packet carries the extracted features and the classification result. The Translator Switch then receives this packet and uses it to perform further **Quality of Experience (QoE) classification** by running a specific, potentially different, model on the incoming classified video packets.

In parallel, the Control Plane oversees model lifecycle management. It handles the offline training of Random Forest models, encodes them into hardware table entries via the `TofinoForestManager`.

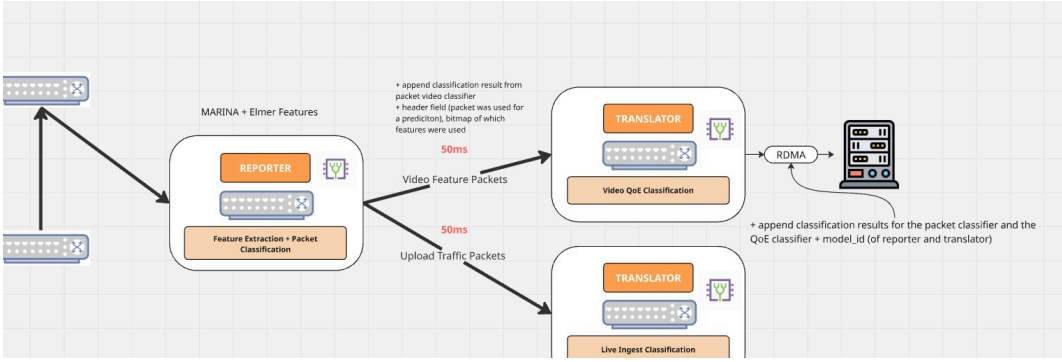


Figure 5: **Architecture of the REPORTER framework, showcasing the interaction between Control Plane model management and Data Plane telemetry generation.**

[2]DTA is a system consisting of several components, each in their own directories.Reporter is a DTA reporter switch. This switch can generate telemetry reports through DTA."

3.2. Data Plane Translator Architecture

The "Translator" is the intelligent core of the proposed in-network telemetry system. While the Reporter component is responsible for aggregating flow statistics, the Translator consumes these statistics, executes a machine learning inference directly within the data plane, and enforces state-dependent routing policies.

Because standard decision trees rely on recursive branching (`if-else` structures) which are incompatible with the strict pipeline constraints of the Intel Tofino ASIC, the P4 program was automatically generated using a custom script (`p4_generator_tna.py`). This generator translates a trained Random Forest model into hardware-compatible Match-Action Tables (MATs).

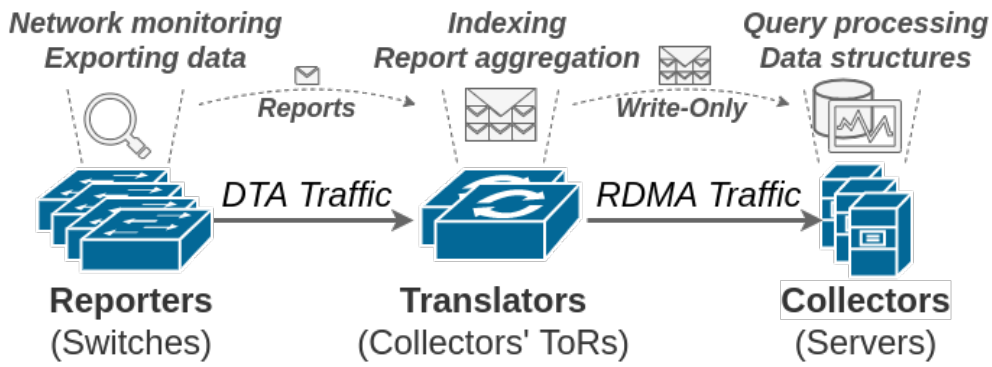


Figure 6: **Translator is a DTA translator switch. This switch will intercept DTA reports and convert these into RDMA traffic. It is in charge of establishing and managing RDMA queue-pairs with the collector server. Source**

3.2.1. Telemetry Parsing and Feature Extraction

The Translator operates primarily on Data-Plane Telemetry Architecture (DTA) packets. Upon ingress, the custom `SwitchIngressParser` inspects the packet headers. If it detects a UDP packet destined for port `40040` with the Marina opcode (`0x05`), it transitions into specialized parsing states to extract the `dta_keyval_static_h` and `keywrite_data_h` headers.

Once parsed, the `FeatureExtraction` control block maps the payload into the `local_metadata_t` structure. While features like `packet_count`, `ps_sum`, and `iat_sum` are extracted directly from the payload, the Translator dynamically calculates `jitter` on the fly. It does this by subtracting the flow's `last_packet_timestamp` (carried in the DTA header) from the switch's local hardware ingress timestamp (`ig_intr_md.ingress_mac_tstamp`).

3.2.2. Random Forest Hardware Mapping

To execute a Random Forest within the Tofino data plane, the `p4_generator_tna.py` script employs two critical design patterns: Ternary Match-Action mapping and Physical Stage Unrolling.

3.2.2.1. Ternary Range Matching (TCAM)

Decision tree nodes evaluate features using inequalities (e.g., $X_{\text{jitter}} < 500$). The Tofino ASIC evaluates inequalities using Ternary Content-Addressable Memory (TCAM). The generator maps the decision thresholds into ternary bitmasks.

Each tree is represented by a set of tables where the match keys include the `current_node` (Exact match) and the four network features (`packet_count`, `ps_sum`, `iat_sum`, `jitter`) as Ternary matches. Depending on the matched rule, the ALU executes either `action_goto_node` to traverse deeper into the tree, or `action_set_leaf` to output a class prediction.

```cpp
table tbl_tree_0_stage_0 {
    key = {
        local_md.classification_variables.current_node_tree0: exact;
        local_md.features.packet_count: ternary;
        local_md.features.ps_sum: ternary;
        local_md.features.iat_sum: ternary;
        local_md.features.jitter: ternary;
    }
    actions = {
        action_goto_node_tree_0;
        action_set_leaf_tree_0;
        NoAction;
    }
}
```

Listing 1: P4 representation of a decision tree node using ternary matching for feature threshold evaluation.

3.2.2.2. Physical Stage Unrolling

The Tofino architecture enforces a strict Directed Acyclic Graph (DAG) for table execution; a table cannot recursively call itself, nor can multiple tables modifying the same metadata execute in the same physical pipeline stage.

To bypass this, the generator "unrolls" the trees. If the Random Forest relies on trees with a maximum depth of 3, the generator creates three distinct tables per tree (e.g., `tbl_tree_0_stage_0`, `tbl_tree_0_stage_1`, `tbl_tree_0_stage_2`). In the `apply` block, these tables are executed sequentially, allowing the packet to traverse from the root to the leaf node across consecutive physical ALU stages.

3.2.3. Voting Mechanism

Because a Random Forest consists of multiple independent decision trees (in this implementation, four trees), their individual predictions must be aggregated to form a final classification. This is handled by a dedicated `tbl_voting` match-action table.

Instead of complex arithmetic averaging, which consumes valuable ALU cycles, the voting logic is pre-computed by the Python generator and hardcoded as exact-match entries. The table takes the four leaf outputs (`tree0_result` through `tree3_result`) as exact match keys and outputs the majority `final_class` alongside a confidence score (the vote tally).

```cpp
table tbl_voting {
    key = {
        local_md.tree0_result : exact;
        local_md.tree1_result : exact;
        local_md.tree2_result : exact;
        local_md.tree3_result : exact;
    }
    actions = { resolve_vote; NoAction; }
    const entries = {
        (0, 0, 0, 0) : resolve_vote(0, 4, 0); // 4 votes for class 0
        (1, 1, 1, 0) : resolve_vote(1, 1, 3); // 3 votes for class 1
        // ... exhaustively generated permutations
    }
}
```

Listing 2: Hardcoded ensemble voting logic utilizing an Exact Match table for instantaneous majority resolution.

3.2.4. State-Dependent Routing and Deparsing

Once the `final_class` is resolved, the Translator uses this result to perform Quality of Experience (QoE) or security routing. This is controlled by the `tbl_classification_action` table.

If the traffic is classified as benign (Class 0), the switch executes `forward_benign` and routes the traffic along its normal path (e.g., egress port 36). However, if the traffic is classified as critical or malicious (Class 1), the switch can execute dynamic mitigation strategies, such as `forward_to_ids` (redirecting the packet to port 64 for deep packet inspection) or `drop_malicious` to quarantine the flow entirely at line rate.

Finally, during the Deparsing stage, the switch does not strip the telemetry. Instead, it recomputes the IPv4 checksum and emits the packet along with a newly appended `classification_h` header. This custom header encapsulates the final class result, the confidence score, and the exact tree vote distribution, allowing downstream analytics servers to verify the Tofino ASIC's in-network inference accuracy.

3.3. Evolution of the Methodology

The initial phase of this research focused on establishing a baseline classification model using "Marina-style" statistical features [15], where packet sizes and inter-arrival times

were aggregated into isolated 50ms time windows using sums and moments. This earlier approach treated every time window as an independent event, relying on a simplistic labeling strategy that categorized network states solely based on the slope of buffer changes (filling versus depleting) without regard for the absolute buffer level.

While this established a foundational correlation between traffic volume and buffer trends, the model lacked temporal context. It could not distinguish between a sudden, transient traffic drop and a sustained outage, nor could it differentiate between a benign buffer drop from a high level and a critical depletion event leading to a video stall.

To address these limitations, the updated pipeline restructured the feature engineering process to introduce **temporal awareness** and stateful logic suitable for deployment on network switches. This shift transitioned the system from a stateless traffic analyzer to a predictive, risk-aware QoE monitor.

3.3.1. Advanced Feature Engineering: Rolling Statistics

A raw 50ms observation is inherently noisy; a single dropped packet or a micro-burst can cause a spike in jitter that does not necessarily reflect the true network state. To smooth these fluctuations and capture the underlying trends (velocity and acceleration) of the traffic, the feature set was augmented using Exponential Moving Averages (EMA).

EMA is particularly suitable for resource-constrained hardware, such as P4-enabled switches, because it allows for "rolling" trend analysis without requiring the storage of a large historical buffer. The switch only needs to store the previous EMA value. The EMA at time t is calculated as:

$$\mathrm{EMA}_t = \alpha \cdot X_t + (1 - \alpha) \cdot \mathrm{EMA}_{t-1}$$

Formula 1: **Exponential Moving Average formula, where X_t is the current observation and α is the smoothing factor.**

Where X_t is the current observation (e.g., jitter or throughput) and α is the smoothing factor. By calculating the EMA over a 20-window span (approx. 1 second), the model gains historical context, allowing it to react to sustained trends rather than transient noise.

Derived Feature	Mathematical Definition
EMA (Trend)	$\mathrm{EMA}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \mathrm{EMA}_{\{t-1\}}$
Lag (Immediate History)	$x_{\mathrm{prev}} = x_{\{t-1\}}$
Rate of Change	$\Delta_{\mathrm{bw}} = \mathrm{BWE}_t - \mathrm{BWE}_{\{t-1\}}$

Table 1: **Temporal features engineered to capture historical context and trends.**

3.3.2. Target Class Definition: Risk-Based QoE State

A supervised learning approach requires well-defined target labels. The updated methodology moved beyond binary slope detection to a risk-based classification scheme.

A "Lookahead" mechanism was implemented to inspect the buffer state 10 steps (500ms) into the future. By comparing the future buffer level ($B_{\{t+10\}}$) with the current level (B_t), a slope was calculated. Crucially, this logic introduced a "Critical" class, defined as a depleting trend occurring when the buffer is **already** below a safety threshold (e.g., 2000ms). This forces the model to prioritize the detection of imminent QoE violations over simple fluctuations.

```python
LOOKAHEAD_STEPS = 10       # 500ms into the future
CRITICAL_BUFFER_MS = 2000  # Safety threshold
DROP_THRESHOLD = -200      # Significant drain rate

# Logic for labeling QoE States
if slope < -200 and buffer < 2000:
    state = "Critical"      # Immediate danger of stalling
elif slope < -200 and buffer >= 2000:
    state = "Safe_Drain"    # Draining, but reserve exists
else:
    state = "Steady"        # Stable or growing buffer
```

Listing 3: Logic used to define the ground-truth QoE target classes.

The resulting class distribution was highly imbalanced, with "Safe_Drain" dominating the dataset (61.3%) and "Critical" instances, the most important cases to detect, representing only 0.04% of the samples.

3.4. Preliminary Modeling and Feature Importance

To establish a baseline for classification performance, a Random Forest Classifier was trained using 150 estimators and a maximum depth of 12. To mitigate the extreme class imbalance, the model utilized `class_weight='balanced'`, which penalizes misclassification of the minority class ("Critical") more heavily.

The input feature vector X for this baseline experiment was reduced to four core metrics: `packet_count`, `ps_sum`, `iat_sum`, and `jitter`.[3]

3.4.1. Performance Evaluation

The model was evaluated on a stratified test set (25% split). As shown in the classification report, the model struggled significantly with the "Critical" class due to the scarcity of samples (only 19 support instances in the test set). While Recall for "Critical" cases was high (0.95), the Precision was near zero, indicating a high false-positive rate.

[3]This initial experiment focused on core traffic metrics to establish a performance baseline. Other features can be used as well, however the trained model might not fit on resource-constrained devices like P4 switches.

Class	Precision	Recall	F1-Score
Critical	0.08	0.95	0.00
Safe_Drain	0.78	0.02	0.03
Steady	0.69	0.69	0.69

Table 2: **Baseline Random Forest performance metrics showing the impact of class imbalance.**

3.4.2. Feature Importance Analysis

Despite the classification challenges, the Random Forest provided insights into feature relevance. The Gini importance scores revealed that traffic volume (`ps_sum`) and instantaneous jitter (`jitter`) were the most significant predictors of the buffer's future state, collectively accounting for over 65% of the model's decision-making power.

Feature	Importance Score
Traffic Volume (`ps_sum`)	0.354
Jitter (`jitter`)	0.302
IAT Sum (`iat_sum`)	0.186
Packet Count	0.158

Table 3: **Feature importance ranking derived from the Random Forest model.**

Finally, the processed feature matrix X and the one-hot encoded target variables Y were exported as NumPy arrays (`.npy`) to facilitate ingestion into deep learning frameworks for subsequent experiments.

4. Implementation

4.1. Dataset Analysis

The dataset used in this thesis originates from Karagkioules et al. paper. The dataset is designed to fill this gap by providing measurements that were simultaneously obtained at the network, transport, and application layers. The data were generated using YouTube's native Android application on two smartphone models at two locations in Europe over a period of more than five months.

At the application layer, a wide range of adaptive streaming parameters was extracted from YouTube's mobile client. This was made possible by a recently introduced Wrapper App [22], which enables remote control and monitoring of YouTube's native Android application. At the transport and network layers, the commonly used tool tcpdump [23] was used to record unfiltered packet logs on both the smartphones and a gateway.

The dataset includes measurements from 8 different scenarios, each representing a different quality and rate limiter. However, Scenario 6 is used since it provides automatic quality selection without any rate limiting, which closely resembles real-world conditions. [24]

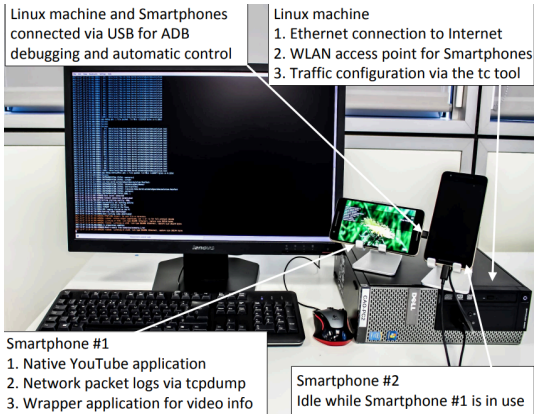


Figure 7: **Hardware setup for regulated and automatic HAS traffic measurements**

4.1.1. Data Ingestion and Parsing

```
(15174 IP(tos 0x0,ttl 55,id 0,offset 0,flags[DF],protoUDP(17),length
1378)74.125.105.91.443>192.168.10.200.4864: UDP, length 135)
```

Listing 4: Sample line from TCPdump network trace log showing a packet with timestamp and length.

The raw data for "Scenario 6" was distributed across a directory structure organized by Video ID and Iteration number. Each iteration contained two distinct data sources: network traffic logs (TCPdump) and application performance metrics (Phone Statistics). To ingest this data, custom parsing functions were developed to handle the semi-structured nature of the logs.

For the network traces, a regular expression was used to extract the timestamp and packet size (length) from each line of the TCPdump output. The regex pattern used to capture these groups is defined below:

```
r'(?P<timestamp>\d+\.\d+).*? length (?P<length>\d+)'
```

Listing 5: Regular expression pattern used to extract packet arrival times and payload lengths.

On the other side, the application logs required a two-step parsing process. The parser first identified lines containing JSON objects embedded within the log text. It then extracted the JSON string, corrected formatting inconsistencies (such as trailing braces), and parsed the object to retrieve the ground-truth labels: Bandwidth Estimate (`bwe`) and Buffer Level (`buffer_level_ms`).

4.1.2. Synchronization and Preprocessing

A critical challenge in this pipeline was aligning the timelines of the two data sources. The application logs recorded timestamps in a timezone differing from the network traces. To rectify this, a one-hour offset was added to the application statistics indices ($t_{\text{stats}} + 1\text{h}$).

Furthermore, to ensure that the feature set captured the context relevant to the video session, the network packets were filtered to a specific time range. This range was defined dynamically as the interval $[t_{\min} - 10\text{s}, t_{\max} + 10\text{s}]$, where $t_{\min}$ and $t_{\max}$ represent the start and end of the application logging period.

4.1.3. Feature Engineering and Windowing

The transformation from raw packet logs to model-ready features involved computing instantaneous metrics followed by a temporal aggregation. The system used a non-overlapping time window of **50ms**.

```python
TIME_WINDOW = '50ms'

#agg_rules defines how to aggregate each metric within the time window
#this includes sums for traffic volume and moments, as well as counts for packet occurrences
features_df = packets_df.resample(TIME_WINDOW).agg(agg_rules)
features_df.columns = [
    'ps_sum', 'packet_count', 'ps2_sum', 'ps3_sum',
    'iat_sum', 'iat2_sum', 'iat3_sum', 'jitter_sum'
]
```

Listing 6: Pandas resampling operation to aggregate packet-level metrics into 50ms time windows.

First, specific metrics were calculated for every packet i in the stream. The Inter-Arrival Time (IAT) was derived as the difference in timestamps between consecutive packets. Jitter was calculated as the absolute difference between consecutive IAT values. To capture

the higher-order statistical properties of the traffic flow, the second and third moments (squares and cubes) were computed for both packet size (PS) and IAT.

$$\mathrm{IAT}_i = t_i - t_{i-1}$$

Formula 2: **Inter-Arrival Time between consecutive packets $i - 1$ and i, where t_i is the arrival timestamp of packet i.**

$$J_i = |\mathrm{IAT}_i - \mathrm{IAT}_{i-1}|$$

Formula 3: **Instantaneous jitter, defined as the absolute difference between consecutive Inter-Arrival Times.**

$$J_i = J_{i-1} + \frac{|\mathrm{IAT}_i - \mathrm{IAT}_{i-1}| - J_{i-1}}{16}$$

Formula 4: **RFC 3550 running jitter estimate, which provides a smoothed, exponentially weighted approximation of jitter as used in RTP streams.**

$$|(J) = \frac{1}{N}\sum_{i=1}^{N}|\mathrm{IAT}_i - \mathrm{IAT}_{i-1}|$$

Formula 5: **Mean jitter over a window of N packets, representing the average deviation of packet spacing.**

$$M_k(\mathrm{PS}) = \sum_{i=1}^{N}\mathrm{PS}_i^k, \quad k \in \{2, 3\}$$

Formula 6: **Second and third statistical moments of packet size within a time window, capturing traffic burstiness and asymmetry.**

$$\mathrm{Throughput}_t = \frac{\sum_{i \in t}\mathrm{PS}_i}{\Delta}t$$

Formula 7: **Estimated throughput per time window t, computed as total byte volume divided by the window duration Δt.**

These values were then resampled into 50ms buckets. The aggregation rules applied to generate the final feature vector X_t for each window t are detailed in Table 1.

Feature Name	Mathematical Derivation	Aggregation Method
Packet Count	Count of events in window	Sum
Traffic Volume (PS)	$\sum \mathrm{PS}_i$	Sum
Traffic Moments	$\mathrm{PS}^2, \mathrm{PS}^3$	Sum (of squares/cubes)
IAT Total	$\Delta t = t_i - t_{i-1}$	Sum
IAT Moments	$\mathrm{IAT}^2, \mathrm{IAT}^3$	Sum (of squares/cubes)
Jitter	$\lvert \mathrm{IAT}_i - \mathrm{IAT}_{i-1} \rvert$	Mean (Sum / Count)

Table 4: **Definition of features extracted per 50ms time window.**

Statistic	Jitter(ms)	Buffer Level (ms)
Count	163,090	163,090
Mean	40.78	105,381.84
Std Dev	441.41	29,514.49
50% (Median)	1.2	120,077.00
75% (Q3)	1.78	122,075.75
Max	9,231.11	129,857.00

Table 5: **Summary statistics for Jitter and Buffer Level across the processed dataset.**

4.1.4. Target Alignment and Dataset Construction

The final stage of the pipeline merged the high-frequency network features with the lower-frequency application labels. Since the application statistics (Buffer Level and BWE) were logged at irregular intervals, a standard join was insufficient. Instead, a `merge_asof` (nearest key) strategy was implemented.

For each 50ms feature window, the system located the nearest application log entry. A strict tolerance limit of **100ms** was enforced; if no application label existed within 100ms of the window's timestamp, the sample was discarded. This ensured that the model would not learn from stale state information.

Post-merge, the dataset underwent a cleaning phase where any rows containing `NaN` values for the target variables were dropped. The final output consisted of a CSV file containing approximately 163,000 samples, encompassing the feature vector, the target labels, and identifiers for the video and iteration.

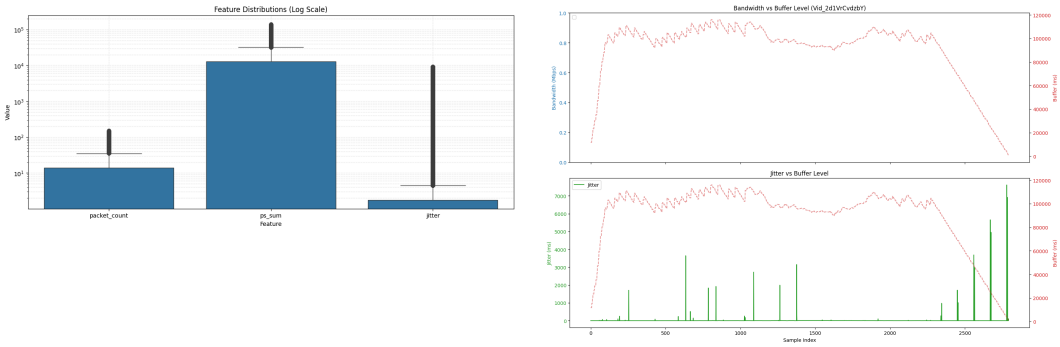


Figure 8: **(Left) Feature distributions (log scale); (Right) Bandwidth vs Buffer Level over time.**

4.2. Model Training via pForest

Following the feature engineering chapter, a simplified version of the pForest [16] architecture was adopted to enable in-network inference on the Intel Tofino switch.

This part relies on the pre-processed datasets (`np_data.npy` (features) and `np_dummies.npy` (labels)) generated in the previous step. These files contain the statistical snapshots required to train the decision trees.

```terminal
├── p4_generator_tna.py
├── randomforest
│   ├── randomForestEncode.py
│   ├── rf_model.py
│   ├── storage
│   │   ├── np_data.npy
│   │   └── np_dummies.npy
│   ├── tester.py
│   └── voting_table_gen.py
```

Listing 7: Files for the Random Forest training logic

4.2.1. The Training Pipeline (`rf_model.py`)

The training logic is found in the `rf_model.py` script. While it imports the standard `scikit` library, the pipeline is constrained by the hardware limitations of the Tofino architecture, specifically the available Match-Action Table (MAT) resources and SRAM capacity.[4]

To ensure the trained model can be compiled to the P4 data plane, the script performs two critical adaptations:

1. **Hardware-Constrained Feature Selection:** Although the initial dataset contains a rich set of features, the Tofino switch cannot efficiently track high-dimensional state vectors for every flow without exhausting memory. Consequently, the training script explicitly reduces the feature space to the four most impactful metrics identified during the preliminary analysis:

[4] Intel Tofino 1 has around 10–12 MB of SRAM, while Tofino 2 explicitly includes 64 MB of shared on-chip packet memory

- Index 0: `packet_count`
- Index 1: `ps_sum` (Traffic Volume)
- Index 2: `iat_sum` (Inter-Arrival Time Sum)
- Index 3: `jitter`

2. **Hyperparameter Tying:** The model complexity is controlled via `num_trees` and `max_depth`. In this context, `max_depth` dictates the number of pipeline stages required for a decision, while `num_trees` impacts the parallel lookup width.

```python
def build_and_train_random_forest(num_trees, max_depth):
    features = np.load(".../storage/np_data.npy", allow_pickle=True)
    labels = np.load(".../storage/np_dummies.npy", allow_pickle=True)

    # REDUCED: Select only top 4 features to fit Tofino constraints
    # Indices: 0=packet_count, 1=ps_sum, 2=iat_sum, 3=jitter
    feature_indices = [0, 1, 2, 3]
    features = features[:, feature_indices]

    rf = RandomForestClassifier(n_estimators=num_trees,
    max_depth=max_depth)
    rf.fit(train_features, train_labels)
    return rf
```

Listing 8: Excerpt from `rf_model.py` demonstrating the loading of exported NumPy data and hardware-specific feature pruning.

4.2.2. Deviation from Original pForest Architecture

A significant architectural distinction exists between the original pForest code [25] and our implementation. The original paper proposes **context-dependent** Random Forests, where distinct models are trained for different phases of a flow (e.g., one model for the first 5 packets, another for packets 6-10). This allows for highly specialized early detection. [16]

In contrast, our implementation deploys a Single Unified Random Forest. This model is trained on the features of **completed** flows (whole-flow statistics) but is applied to live traffic where features accumulate incrementally.

4.2.3. Automated P4 Code Generation (`p4_generator_tna.py`)

To bridge the gap between the trained model and the hardware implementation, the system uses `p4_generator_tna.py`. This script builds the P4 program, ensuring compliance with Tofino's strict resource constraints,such as Ternary Content Addressable Memory (TCAM) usage and stage limitations.

The pForest script is managed by a single main function, `generate_full_p4_code`. This function accepts the model's structural hyperparameters (`num_trees`, `max_depth`) and sequentially assembles the pipeline components: identifying necessary headers, generating the "physically unrolled" match-action tables for the decision trees, and constructing the static voting logic.

```python
def generate_full_p4_code(num_trees=4, max_depth=3):
    """Generate complete P4 code optimized for Tofino"""

    feature_names = ['packet_count', 'ps_sum', 'iat_sum', 'jitter']

    p4_code = '''/* Generated P4_16 for Tofino */
#include <core.p4>
#include <tna.p4>
'''
    # Modular generation of pipeline stages
    p4_code += generate_p4_headers()
    p4_code += generate_parser()
    p4_code += generate_feature_extraction()
    # Generates the Random Forest logic (Tables + Actions)
    p4_code += generate_classification_control(num_trees,
    feature_names, max_depth)
    p4_code += generate_forwarding_logic_and_ingress()
    p4_code += generate_deparser()
    p4_code += generate_pipeline()

    return p4_code
```

Listing 9: The main driver function of the generator script, demonstrating the modular assembly of the P4 pipeline.

The final output is written to `rf_classifier_dta_marina.p4`, producing a fully compilable network program that integrates the specific Random Forest logic into the switch's standard forwarding behavior.

4.2.4. Control Plane Configuration: Forwarding

In a Software-Defined Networking (SDN) architecture using P4, compiling and loading the program onto the Tofino ASIC only defines the pipeline's structure. Upon initialization, the match-action tables within the data plane are entirely empty, meaning the switch will default to dropping all incoming packets. To establish connectivity within the hardware-in-the-loop testbed, a control plane application is required to populate the forwarding tables.

For this evaluation, a Python script was developed utilizing the Barefoot Runtime (BFRT) gRPC interface. Unlike full deployment scripts that initialize the entire ML pipeline, telemetry thresholds, and routing logic simultaneously, this script was intentionally decoupled to focus exclusively on the `tbl_forward` table.

4.2.5. Isolated Forwarding Table Configuration

The decision to isolate the forwarding logic was driven by the need for modularity during testing. By removing the overhead of configuring the entire P4 pipeline, this script allows for rapid, incremental updates to the routing topology without resetting the switch's telemetry or machine learning states. This is particularly advantageous during debugging or when switching between the "Reporter" and "Translator" testing phases, as

it guarantees that basic Layer 3 forwarding remains intact regardless of the data plane's experimental features.

4.2.6. BFRT gRPC Implementation

The script connects to the Tofino's BFRT server on `localhost:50052` and binds to the deployed pipeline profile (`p4_marina_reporter`). It then targets the `SwitchIngress.tbl_forward` table, retrieving the dynamically assigned action IDs directly from the compiler's output, which ensures robustness against P4 code modifications.

```python
# Construct the Match Key (Destination IP)
key =[
    tbl_forward.make_key([
        gc.KeyTuple('hdr.ipv4.dstAddr',
                    gc.ipv4_to_bytes('192.168.100.1'),
                    prefix_len=32)
    ])
]

# Construct the Action Data (Egress Port and MAC rewrite)
data = [
    tbl_forward.make_data([
        gc.DataTuple('port', 16), # Logical Port 16 (Physical 24)
        gc.DataTuple('dst_mac', mac_to_bytes('10:70:fd:30:80:d1')),
        gc.DataTuple('src_mac', mac_to_bytes('D0:77:CE:2B:20:54')),
    ], action_forward)
]

# Insert into Tofino hardware
tbl_forward.entry_add(target, key, data)
```

Listing 10: BFRT gRPC snippet demonstrating the insertion of a match-action entry into the Tofino hardware.

4.2.7. Table Logic and Topology Integration

The hardware testbed consists of the TRex traffic generator communicating with the Tofino switch via an intermediate Arista switch. Because the TRex server generates traffic destined for `192.168.100.1`, the Tofino switch must correctly identify these packets, assign them to the correct egress port, and rewrite the Ethernet headers to ensure the Arista switch forwards them back to the TRex RX interface.

The inserted entry implements Exact Match logic on the IPv4 destination address. When a packet matches, the action parameters (`port`, `dst_mac`, and `src_mac`) are applied natively at line rate by the switch's ALUs. Table 3 outlines the exact mapping programmed by the control plane script.

Component	Parameter	Configured Value
Match Key	`hdr.ipv4.dstAddr`	`192.168.100.1/32` (Exact Match)
Action Data	`port`	`16` (Logical Egress Port to TRex)
Action Data	`src_mac`	`D0:77:CE:2B:20:54` (Tofino Egress MAC)
Action Data	`dst_mac`	`10:70:fd:30:80:d1` (TRex RX Interface MAC)

Table 6: **Forwarding rules populated in the data plane by the control plane script.**

By successfully executing this script, the control plane ensures that any packet, whether it is raw telemetry traffic or an evaluated packet returning from the QoE ML models, is correctly encapsulated and routed back to the TRex server for statistical analysis and PCAP capture. Furthermore, the inclusion of exception handling in the script guarantees that connection failures or missing dependencies are gracefully reported, significantly simplifying the operational workflow of the testbed.

4.2.8. Full Control Plane Script

The following code block contains the complete Python script used to interact with the Barefoot Runtime (BFRT) gRPC interface. This script is responsible for pushing the exact match forwarding rules to the Tofino switch's data plane.

```python
"""
Insert Forward Table Entries
============================
This script inserts entries into the forward table for the Marina Reporter P4 program.

Physical Topology:
  TRex Port 0 (10:70:fd:30:80:d0) --> Arista --> Tofino Port 23 (Logical 8, MAC D0:77:CE:2B:20:50)
  Tofino Port 24 (Logical 16, MAC D0:77:CE:2B:20:54) --> Arista --> TRex Port 1 (10:70:fd:30:80:d1)
"""

import sys
import glob
import os

# CONFIGURATION

# Tofino logical port numbers (from bf_shell "pm show" D_P column)
TOFINO_EGRESS_PORT = 16    # Logical port 16 (Physical 24) - sends to TRex

# Tofino port MACs
TOFINO_EGRESS_MAC = "D0:77:CE:2B:20:54"  # Port 24 / Logical 16

# TRex interface MACs
TREX_RX_MAC = "10:70:fd:30:80:d1"  # TRex Port 1 (RX from Tofino)

# Traffic IPs
TRAFFIC_DST_IP = "192.168.100.1"

# BF Runtime gRPC setup
sde_install = os.environ.get('SDE_INSTALL', '/usr/local/sde')
bfrt_location = '{}/lib/python*/site-packages/tofino'.format(sde_install)

try:
    found_paths = glob.glob(bfrt_location)
    if found_paths:
        sys.path.append(found_paths[0])
    else:
        print(f"WARNING: Could not find SDE python libraries at {bfrt_location}")
    import bfrt_grpc.client as gc
```

```python
except ImportError as e:
    print(f"CRITICAL: Could not import bfrt_grpc. Error: {e}")
    print(f"SDE_INSTALL: {sde_install}")
    sys.exit(1)


def mac_to_bytes(mac_str):
    """Convert MAC string like 'AA:BB:CC:DD:EE:FF' to bytearray."""
    return bytearray(int(x, 16) for x in mac_str.split(':'))


def main():
    print("=" * 60)
    print("Insert Forward Table Entries")
    print("=" * 60)

    print("\nConnecting to BFRT Server (localhost:50052)...")
    try:
        interface = gc.ClientInterface('localhost:50052', client_id=0, device_id=0)
    except Exception as e:
        print(f"Connection failed: {e}")
        return

    try:
        interface.bind_pipeline_config('p4_marina_reporter')
    except Exception as e:
        print(f"WARNING: bind_pipeline_config failed: {e}")

    bfrt_info = interface.bfrt_info_get('p4_marina_reporter')

    # Target for P4 tables (all pipes)
    target = gc.Target(device_id=0, pipe_id=0xffff)

    # ============================================================================
    # Get Forward Table
    tbl_forward = bfrt_info.table_get('SwitchIngress.tbl_forward')

    # Resolve action name (varies by SDE version / P4 compilation)
    try:
        action_forward = next(iter(tbl_forward.info.action_dict.keys()))
    except Exception:
        print("CRITICAL: Could not resolve action name for forward table.")
        return

    # ============================================================================
```

```
    # Insert Forward Table Entry
    print("Inserting Forwarding Table Entry...")
    tbl_forward.entry_add(
        target,
        [tbl_forward.make_key([
            gc.KeyTuple('hdr.ipv4.dstAddr',
            gc.ipv4_to_bytes(TRAFFIC_DST_IP), prefix_len=32)
        ])],
        [tbl_forward.make_data([
            gc.DataTuple('port', TOFINO_EGRESS_PORT),
            gc.DataTuple('dst_mac', mac_to_bytes(TREX_RX_MAC)),
            gc.DataTuple('src_mac', mac_to_bytes(TOFINO_EGRESS_MAC)),
        ], action_forward)]
    )
    print(f"  {TRAFFIC_DST_IP}/32 -> port {TOFINO_EGRESS_PORT} "
          f"(dst={TREX_RX_MAC}, src={TOFINO_EGRESS_MAC})")

    print("\n" + "=" * 60)
    print("FORWARD TABLE ENTRY INSERTED")
    print("=" * 60)


if __name__ == "__main__":
    main()
```

5. Evaluation

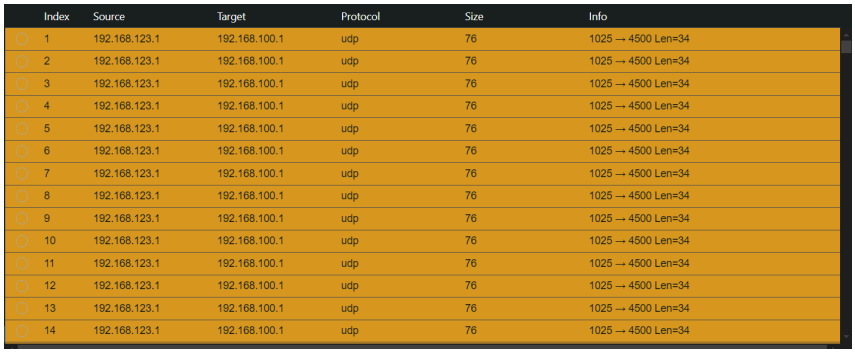
5.1. System Evaluation and Translator Testing

To evaluate the performance and accuracy of the in-network Quality of Experience (QoE) classification model, a hardware-in-the-loop testbed was utilized. The topology consists of a TRex software traffic generator connected to an Intel Tofino programmable switch. The evaluation was conducted in a two-phase approach: first generating the necessary telemetry data, and second, testing the Translator's inference and routing capabilities.

5.1.1. Phase 1: Telemetry Data Generation (Reporter)

The Translator component relies on Data-Plane Telemetry Architecture (DTA) packets to perform inference. Because these packets do not exist in the raw public datasets, they had to be synthetically generated using the actual P4 hardware.

To achieve this, the TRex server was configured to replay raw TCPdump logs from Scenario 6 of the Würzburg QoE dataset into the Tofino switch. The switch, loaded with the "Reporter" P4 program, processed this traffic, forwarded it back to the server, and generated a dedicated DTA telemetry packet every 100ms containing the aggregated network features. These DTA packets were captured into a PCAP file (`trex_tofino_reply.pcap`) to serve as the ground-truth input for the subsequent Translator evaluation.



Index	Source	Target	Protocol	Size	Info
1	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
2	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
3	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
4	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
5	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
6	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
7	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
8	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
9	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
10	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
11	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
12	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
13	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
14	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34

Figure 9: **Pcap file inspected with pcapviewer extension (VsCode)**

5.1.2. Phase 2: Translator Inference Testing

In the second phase, the Tofino switch was reprogrammed with the "Translator" P4 program, which includes the embedded Random Forest ML tables. The objective was to replay the previously captured DTA packets through the Translator to verify if the switch could correctly classify the QoE state in real-time, append the classification result to the packet header, and execute state-dependent routing decisions.

To automate this, a custom Python testing framework (`traffic_gen_translator.py`) was developed utilizing the TRex Stateless (STL) API and the Scapy packet manipulation library.

5.1.3. Packet Manipulation and Topology Adaptation

When replaying PCAP files in a live network testbed, the original Layer 2 addresses are no longer valid for the current topology. The script utilizes Scapy to load the DTA packets

(filtered by UDP port 40040) and rewrites the Source and Destination MAC addresses. The Source MAC is updated to the TRex TX interface, and the Destination MAC is set to the Tofino switch.

Crucially, because the IP and UDP headers are modified, the script explicitly deletes the existing checksums, forcing Scapy to recalculate them before transmission. This ensures the packets are not dropped by the switch due to checksum validation failures.

```python
for pkt in packets:
    if UDP in pkt and pkt[UDP].dport == DTA_UDP_PORT:
        # Rewrite MACs for the live testbed topology
        pkt[Ether].src = TREX_TX_MAC
        pkt[Ether].dst = TOFINO_MAC

        # Force checksum recalculation after header modification
        if IP in pkt: del pkt[IP].chksum
        del pkt[UDP].chksum
```

Listing 11: Scapy packet manipulation logic to adapt saved DTA PCAPs for live replay.

5.1.4. Timing Preservation

Machine learning models analyzing network traffic, especially those utilizing features like jitter, Inter-Arrival Time (IAT), and Exponential Moving Averages (EMA), are highly sensitive to temporal dynamics. Replaying the PCAP file as a single high-speed burst would compress the time windows and invalidate the ML classification.

To prevent this, the script computes the exact microsecond gap between consecutive packets from the original PCAP timestamps. It then utilizes TRex's `STLStream` objects, specifically utilizing the Inter-Stream Gap (`isg`) parameter and the `next` attribute. This creates a chained loop of packets that perfectly mimics the original 100ms emission rate of the Reporter program.

5.1.5. Overcoming Testbed Artifacts: MAC Learning

A significant challenge encountered during testing involved intermediate network equipment (e.g., Arista switches) located between the TRex server and the Tofino switch. Because the TRex RX port only receives traffic and does not natively transmit, the intermediate switch fails to learn the RX port's MAC address, resulting in dropped or flooded return traffic.

To resolve this, a "Keepalive" mechanism was implemented. Before initiating the DTA traffic burst, the script instructs TRex to transmit dummy broadcast UDP packets from the RX port. This forces the intermediate switch to populate its MAC address table (FDB), ensuring a clean and reliable return path for the packets evaluated by the Tofino switch.

```python
# Keepalive on RX port (CRITICAL - prevents intermediate switch drops)
keepalive_pkt = (
    Ether(src=TREX_RX_MAC, dst="ff:ff:ff:ff:ff:ff") /
    IP(src=TREX_RX_IP, dst="255.255.255.255") /
    UDP(sport=12345, dport=12345) / Raw(load="keepalive")
)
keepalive_stream = STLStream(
    packet=STLPktBuilder(pkt=keepalive_pkt),
    mode=STLTXCont(pps=5)
)
c.add_streams(keepalive_stream, ports=[rx_port])
c.start(ports=[rx_port], force=True)
```

Listing 12: Implementation of the Keepalive mechanism to trigger MAC learning on intermediate switches.

5.1.6. Execution and Capture

With the topology established and timing configured, the TRex server transitions the RX port into a service mode and begins capturing traffic. The DTA streams are transmitted from the TX port for a defined duration (10 seconds). As the packets pass through the Tofino switch, the P4 Translator program applies the Random Forest classification, appends the resulting QoE state to the packet, and routes it back.

The script continually polls and displays the real-time TX/RX statistics to verify that no packets are dropped during inference. Finally, the test concludes by halting transmission, stopping the capture buffer, and saving the evaluated packets into a new output file (`trex_tofino_translator_reply.pcap`) for offline validation of the model's accuracy.

5.1.7. Telemetry Generation Results

Following the execution of the hardware-in-the-loop tests, the captured traffic (`trex_tofino_reply.pcap`) was parsed offline to verify the proper generation of Data-Plane Telemetry Architecture (DTA) packets by the Tofino switch. The parser processed approximately 28.5 MB of traffic, successfully filtering and isolating the telemetry reports from the background traffic.

Traffic Metric	Count
Total Captured Packets	20,263
Background / Other Packets	20,228
DTA Telemetry Reports generated	**35**

Table 7: **Summary of the captured PCAP file during the Reporter testing phase.**

Of the 20,263 packets captured, the switch successfully emitted 35 dedicated DTA telemetry reports (one every 100ms as configured). To validate the hardware logic, the payload of the telemetry reports was fully decoded. Table 2 details the specific fields extracted from Report #1.

The values confirm that the Tofino switch correctly tracked the stateful flow metrics, such as accumulated packet counts, Inter-Arrival Time (IAT) sums, and jitter directly within the data plane. Additionally, the report appends the machine learning classification outcome and a bitmap indicating which specific features were utilized during the Random Forest inference step.

Telemetry Field	Decoded Hardware Value
L2 MAC Address	`d0:77:ce:2b:20:54 -> 10:70:fd:30:80:d1`
L3 IP Address	`192.168.123.1 -> 192.168.100.2`
L4 Protocol	`UDP 49374 -> 40040 (DTA)`
— DTA Header Metadata —	
Opcode	`0x05`
Redundancy Level	`1`
Telemetry Key (Flow ID)	`2980564395`
Monitored Flow	`192.168.123.1:1025 -> 192.168.100.1:4500 (UDP)`
Packet Count	198
Last Packet Timestamp	850,630,301
Sum of IAT	99,952,039
Sum of IAT^2 / IAT^3	0 / 0
Sum of Packet Size	272,844
Sum of Packet $Size^2$ / $Size^3$	0 / 0
Jitter	2,001,694
Classification Result	**`0`** (NON-VIDEO)
Used Features Bitmap	`0x0000000d`

Table 8: **Detailed breakdown of DTA Telemetry Report #1, showcasing the stateful metrics computed by the Tofino hardware and the resulting QoE classification.**

5.2. Benchmarking and Performance Analysis

This section presents the results of the benchmarking framework developed to evaluate the practical limits of in-network QoE prediction. The automated pipeline, driven by the `code/benchmark.py` script, trains Random Forest models with varying hyperparameters, translates them into P4 code, and utilizes the Intel SDE compiler to evaluate architectural resource consumption.

5.2.1. Feature Scaling and Hardware Constraints

The first major experiment evaluates the impact of the number of telemetry features (k) on both Machine Learning (ML) performance and hardware resource utilization. As shown in the benchmarking plots, increasing the number of features allows the Random Forest to capture more complex patterns in the traffic, improving accuracy and the F1 score.

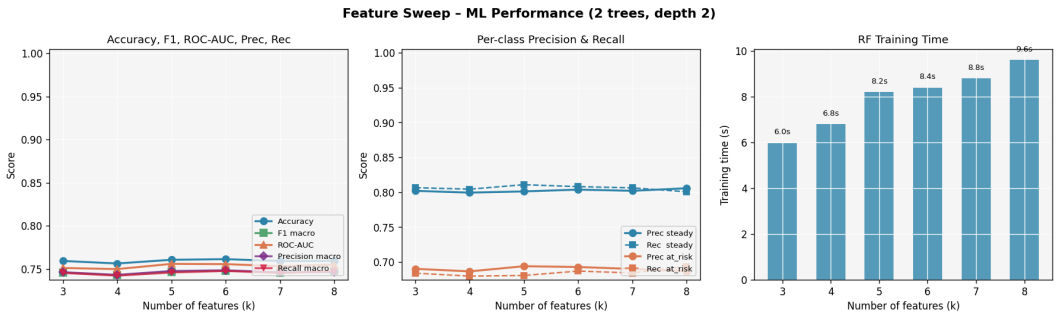


Figure 10: **ML performance metrics across a varying number of features (k).**

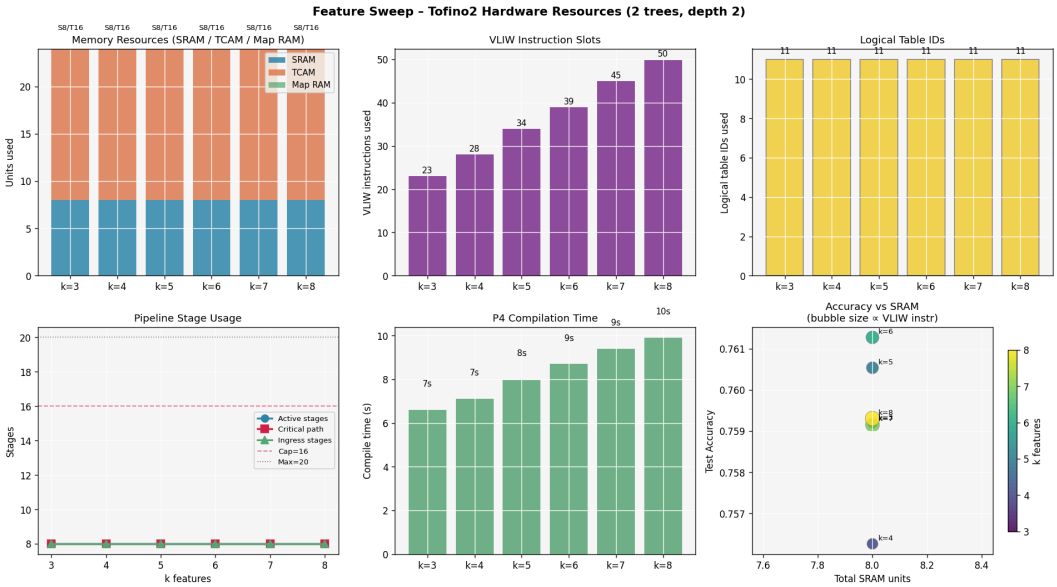


Figure 11: **Hardware resource consumption (SRAM, TCAM, VLIW instructions, and pipeline stages) relative to the number of features.**

However, this improvement in ML metrics comes at a steep cost in terms of hardware resources. The compiler reports indicate that SRAM, TCAM, and specifically VLIW instruction slots scale rapidly as k increases. The physical limit of 20 pipeline stages on the

Tofino 2 architecture becomes a hard bottleneck, as demonstrated in the stage utilization heatmaps.

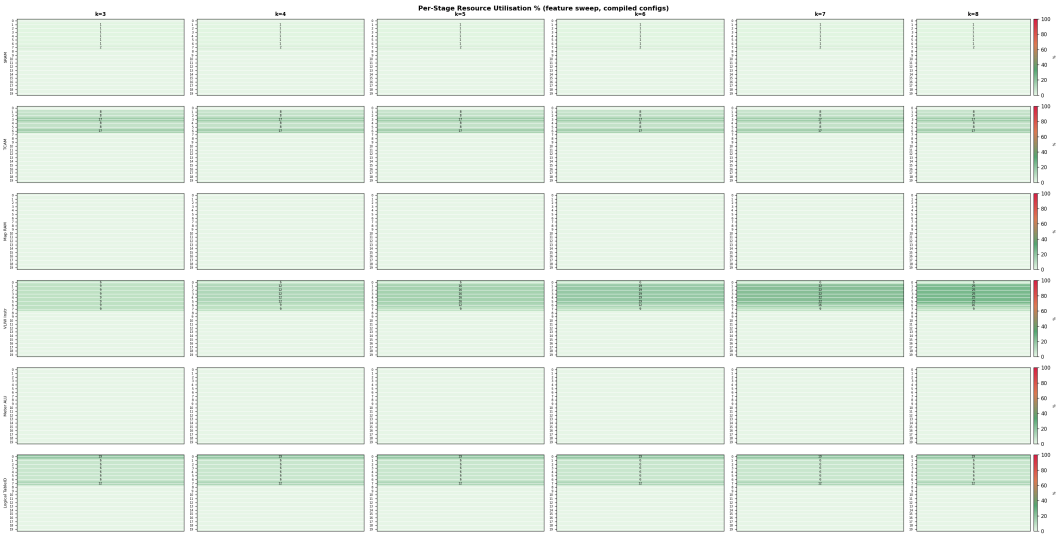


Figure 12: **Per-stage resource utilization heatmaps showing the rapid consumption of available pipeline stages.**

5.2.2. Tree and Depth Configuration Sweep

The second experiment systematically sweeps the number of trees and the maximum depth of the Random Forest. While larger and deeper forests naturally yield higher test accuracies and better recall, they strain the P4 compiler's ability to fit the logic within the limited hardware stages.

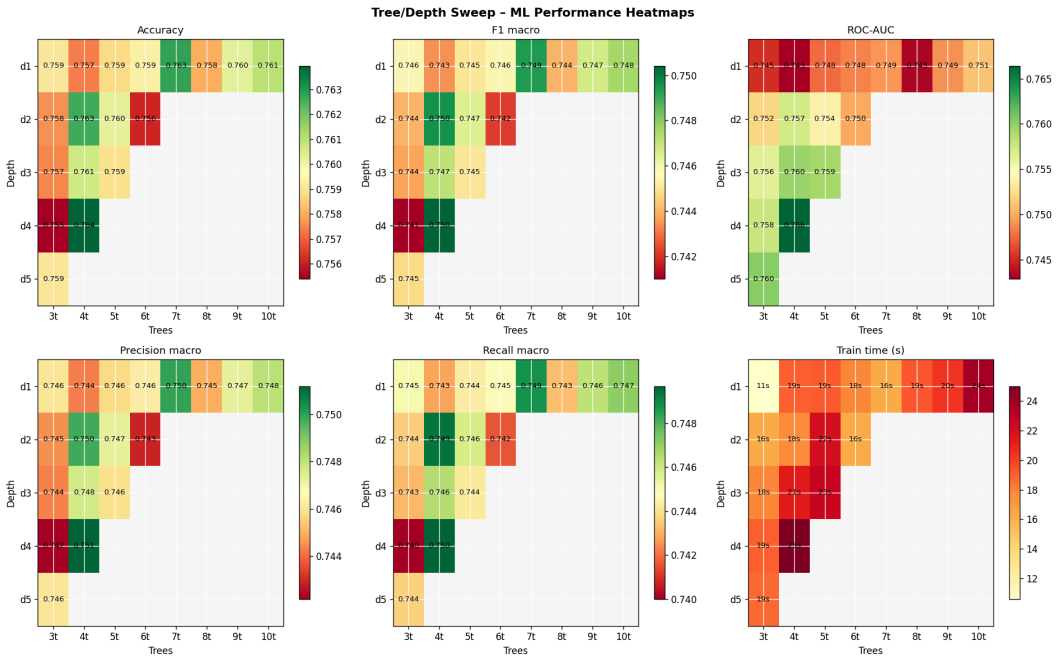


Figure 13: **Heatmaps correlating the number of trees and maximum depth with overall ML performance.**

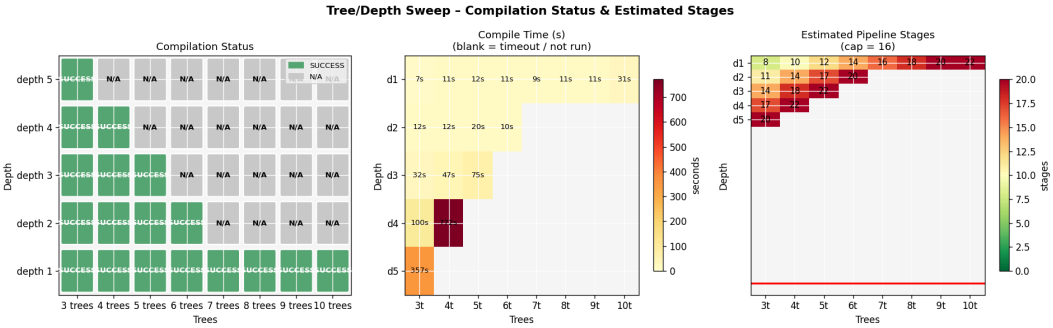


Figure 14: **Compilation status and estimated pipeline stages for various tree and depth configurations. Configurations exceeding the 20-stage limit fail to compile.**

The compilation status grid highlights the operational boundaries of in-network ML. Configurations that exceed a certain complexity threshold either time out during compilation or fail completely due to the exhaustion of active pipeline stages and logical table IDs.

5.2.3. Feature Importance and Resource Overview

To optimize the models for deployment, it is crucial to understand which features contribute most to the classification. The Gini feature importances confirm that a small subset of the MARINA telemetry features provides the majority of the predictive power.

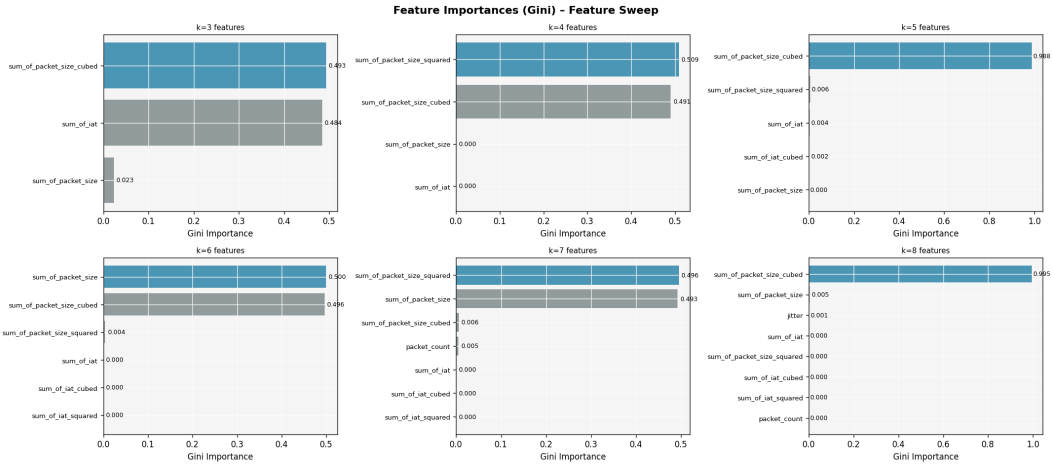


Figure 15: **Feature importances calculated during the feature sweep experiment.**

The aggregated resource overview illustrates the complex multi-dimensional trade-off between test accuracy, total SRAM allocated, active pipeline stages, and VLIW instructions. It proves that by carefully tuning the hyperparameters, an optimal configuration can be found that maximizes QoE prediction accuracy while remaining strictly within the hardware bounds of the Tofino 2 ASIC.

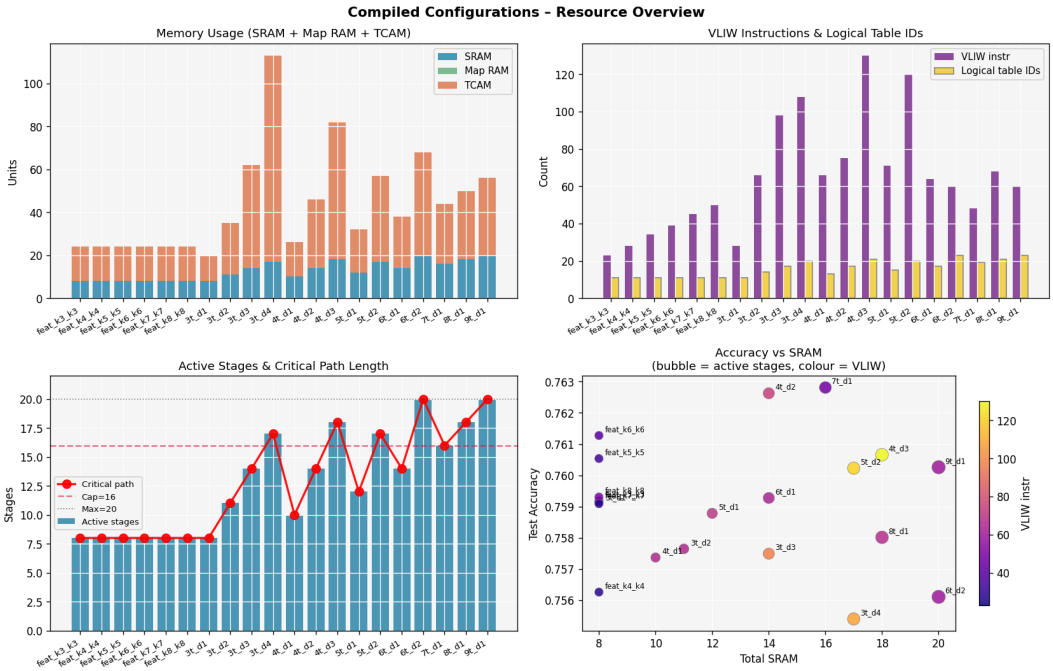


Figure 16: **Combined resource overview mapping accuracy against SRAM usage and active stages.**

5.3. Conclusion

In conclusion, this work bridges the gap between high-level machine learning models and low-level data plane programming. By translating decision trees into match-action tables and evaluating them systematically, this thesis confirms that line-rate, in-network QoE prediction is a viable and highly efficient approach. The benchmarking suite confirms that while modern programmable switches impose strict memory and stage limitations, intelligent feature selection and hyperparameter tuning allow robust ML models to operate at terabit speeds directly within the network infrastructure.

Literature and Bibliography

[1] S. Wassermann and et al., “ViCrypt to the Rescue: Real-Time, Machine-Learning-Driven Video-QoE Monitoring for Encrypted Streaming Traffic.”

[2] O. Izima, R. de Fréin, and M. Davis, “Video Quality Prediction Under Time-Varying Loads.” Nicosia, Cyprus, 2018.

[3] M. T. Vega, C. Perra, and A. Liotta, “Resilience of Video Streaming Services to Network Impairments,” vol. 64. pp. 220–234, 2018.

[4] Cisco Systems, “Cisco Visual Networking Index: Forecast and Trends, 2017–2022.” Accessed: Oct. 01, 2021. [Online]. Available: https://www.cisco.com/c/dam/m/en_us/solutions/service-provider/vni-forecast-highlights/pdf/Global_Device_Growth_Traffic_Profiles.pdf

[5] International Telecommunication Union, “Subjective Video Quality Assessment Methods for Multimedia Applications.” [Online]. Available: https://www.itu.int/rec/T-REC-P.910-200804-I

[6] S. Chikkerur, V. Sundaram, M. Reisslein, and L. J. Karam, “Objective Video Quality Assessment Methods: A Classification, Review, and Performance Comparison.”

[7] International Telecommunication Union, “Reference Guide to Quality of Experience Assessment Methodologies.” [Online]. Available: https://www.itu.int/rec/T-REC-G.1011-201607-I/en

[8] A. Bentaleb, B. Taani, A. C. Begen, C. Timmerer, and R. Zimmermann, “A Survey on Bitrate Adaptation Schemes for Streaming Media over HTTP.”

[9] Z. Duanmu, K. Zeng, K. Ma, A. Rehman, and Z. Wang, “A Quality-of-Experience Index for Streaming Video,” vol. 11. 2016.

[10] Q. Huynh-Thu and M. Ghanbari, “Scope of Validity of PSNR in Image/Video Quality Assessment.”

[11] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image Quality Assessment: From Error Visibility to Structural Similarity,” vol. 13. 2004.

[12] S. Wolf and M. Pinson, “Reference Algorithm for Computing Peak Signal to Noise Ratio (PSNR) of a Video Sequence with a Constant Delay.” [Online]. Available: https://www.its.bldrdoc.gov/

[13] R. U. Mustafa, D. Moura, and C. E. Rothenberg, “Machine Learning Approach to Estimate Video QoE of Encrypted DASH Traffic in 5G Networks,” in *2021 IEEE Statistical Signal Processing Workshop (SSP)*, Rio de Janeiro, Brazil, 2021, pp. 586–589. doi: 10.1109/SSP49050.2021.9513804.

[14] M. Sidorov, R. Birman, O. Hadar, and A. Dvir, “Estimating QoE from Encrypted Video Conferencing Traffic.”

[15] L. Froschauer, J. Langlet, and A. Kassler, “Direct Feature Access – Scaling Network Traffic Feature Collection to Terabit Speed.”

[16] C. Busse-Grawitz, R. Meier, A. Dietmüller, T. Bühler, and L. Vanbever, "pForest: In-Network Inference with Random Forests." [Online]. Available: https://arxiv.org/abs/1909.05680

[17] C. Zheng and N. Zilberman, "Planter: Seeding Trees Within Switches."

[18] G. Xie *et al.*, "Empowering In-Network Classification in Programmable Switches by Binary Decision Tree and Knowledge Distillation."

[19] C. Zheng, X. Hong, D. Ding, S. Vargaftik, Y. Ben-Itzhak, and N. Zilberman, "In-Network Machine Learning Using Programmable Network Devices: A Survey."

[20] C. Zheng *et al.*, "IIsy: Hybrid In-Network Classification Using Programmable Switches."

[21] A. Algazinov, J. Chandra, and M. Laing, "INSIGHT: A Survey of In-Network Systems for Intelligent, High-Efficiency AI and Topology Optimization."

[22] M. Seufert and et al., "A Wrapper for Automatic Measurements with YouTube's Native Android App." June 2018.

[23] The tcpdump Group, "tcpdump Packet Analyzer v. 4.7.4 (libpcap v. 1.7.4)." [Online]. Available: http://www.tcpdump.org/

[24] T. Karagkioules *et al.*, "A Public Dataset for YouTube's Mobile Streaming Client."

[25] roxennnn, "pForest Source Code." [Online]. Available: https://github.com/roxennnn/pforest/tree/master/src

List of Figures

Figure 1 a network architecture diagram illustrating the data flow from streaming servers to client devices, while highlighting specific stages for measuring Quality of Service (QoS), Quality of Delivery (QoD), and Quality of Experience (QoE). Source II
Figure 2 Video effect of the devices on traffic is more pronounced because of the introduction of Ultra-High-Definition (UHD), or 4K, video streaming. Source . . 1
Figure 3 MPEG DASH is a scenario where a video player dynamically switches between different quality levels of media segments delivered over HTTP to adapt to fluctuating network conditions for smooth playback. 2
Figure 4 The general architecture of a P4 switch. [18] 4
Figure 5 Architecture of the REPORTER framework, showcasing the interaction between Control Plane model management and Data Plane telemetry generation. 5
Figure 6 Translator is a DTA translator switch. This switch will intercept DTA reports and convert these into RDMA traffic. It is in charge of establishing and managing RDMA queue-pairs with the collector server. Source 6
Figure 7 Hardware setup for regulated and automatic HAS traffic measurements 12
Figure 8 (Left) Feature distributions (log scale); (Right) Bandwidth vs Buffer Level over time. 16
Figure 9 Pcap file inspected with pcapviewer extension (VsCode) 24
Figure 10 ML performance metrics across a varying number of features (k). 28
Figure 11 Hardware resource consumption (SRAM, TCAM, VLIW instructions, and pipeline stages) relative to the number of features. 28
Figure 12 Per-stage resource utilization heatmaps showing the rapid consumption of available pipeline stages. 29
Figure 13 Heatmaps correlating the number of trees and maximum depth with overall ML performance. 29
Figure 14 Compilation status and estimated pipeline stages for various tree and depth configurations. Configurations exceeding the 20-stage limit fail to compile. . 30
Figure 15 Feature importances calculated during the feature sweep experiment. 30
Figure 16 Combined resource overview mapping accuracy against SRAM usage and active stages. 31

List of Tables

Table 1 Temporal features engineered to capture historical context and trends. 9
Table 2 Baseline Random Forest performance metrics showing the impact of class imbalance. 11
Table 3 Feature importance ranking derived from the Random Forest model. 11
Table 4 Definition of features extracted per 50ms time window. 15
Table 5 Summary statistics for Jitter and Buffer Level across the processed dataset. . . . 15
Table 6 Forwarding rules populated in the data plane by the control plane script. 20
Table 7 Summary of the captured PCAP file during the Reporter testing phase. 26
Table 8 Detailed breakdown of DTA Telemetry Report #1, showcasing the stateful metrics computed by the Tofino hardware and the resulting QoE classification. 27

List of Formulas

Formula 1 Exponential Moving Average formula, where X_t is the current observation and α is the smoothing factor. 9
Formula 2 Inter-Arrival Time between consecutive packets $i-1$ and i, where t_i is the arrival timestamp of packet i. 14
Formula 3 Instantaneous jitter, defined as the absolute difference between consecutive Inter-Arrival Times. 14
Formula 4 RFC 3550 running jitter estimate, which provides a smoothed, exponentially weighted approximation of jitter as used in RTP streams. 14
Formula 5 Mean jitter over a window of N packets, representing the average deviation of packet spacing. 14
Formula 6 Second and third statistical moments of packet size within a time window, capturing traffic burstiness and asymmetry. 14
Formula 7 Estimated throughput per time window t, computed as total byte volume divided by the window duration Δt. 14

List of Abbreviations

AI **– Artificial Intelligence**

ASIC **– Application-Specific Integrated Circuit**

BFRT **– Barefoot Runtime**

BWE **– Bandwidth Estimate**

DAG **– Directed Acyclic Graph**

DTA **– Data-Plane Telemetry Architecture**

EMA **– Exponential Moving Average**

IAT **– Inter-Arrival Time**

IP **– Internet Protocol**

ITU **– International Telecommunication Union**

KPI **– Key Performance Indicator**

MAC **– Media Access Control**

MAT **– Match-Action Table**

ML **– Machine Learning**

PCAP **– Packet Capture**

PSNR **– Peak Signal-to-Noise Ratio**

QoE **– Quality of Experience**

RDMA **– Remote Direct Memory Access**

RF **– Random Forest**

SDN **– Software-Defined Networking**

SSIM **– Structural Similarity Index**

TCAM **– Ternary Content-Addressable Memory**

UDP **– User Datagram Protocol**

UHD **– Ultra-High-Definition**

VQA **– Video Quality Assessment**